SW Engineering CSC648-848-05 Spring 2023

StudentSources

Team #04 “Fantastic Four”
Saru Shrestha (sshrestha11@sfsu.edu)
Kilian Kistenbroker
Elias Abay
Parth Panchal
Joshua Del Rosario
Wilson Yin
Omar Thiongane

Milestone 2 - Version 2
April 26, 2023

History Table:

M4v1T4	04/18/2023
M3v2T4	04/15/2023
M3v1T4	04/27/2023
M2v2T4	04/26/2023
M2v1T4	03/30/2023
M1v2T4	04/19/2023
M1v1T4	03/02/2023

Table of Contents

Data Definitions.....3
Prioritized Functional Requirements..... 6
UI Mockups and Storyboards..... 12
High level database architecture and organization..... 21
Database Organization..... 21
Media Storage..... 27
Search/filter architecture and implementation..... 27
High Level APIs and Main Algorithms..... 28
High Level UML Diagrams.....29
High Level Application Network and Development Diagrams.....30
Key Risks in Current Work.....31
Project Management.....33
List of Team Member Contributions.....34
Team Member Contribution Score.....35

Data Definitions

General User: A general user is one who doesn't have permission to view any of the approved resources. As they also automatically lack an account on StudentSources, general users do not necessarily need to either log in nor do they have to register and have access to basic features. However, if they choose to do so, general users can sign up by providing an email address, password, and to make sure that they agree to the terms and conditions.

Registered User: Registered users are essentially users who have access to an account on StudentSources. Unlike general users, registered users have the capability to view resources that have been approved as well as have the capability to upload their own. Also, registered users are able to open their own file via creating separate tabs as long as they are logged in and registered into the system.

Account: An account is the one thing that differs between general users and registered users. Accounts store various forms of data belonging to those who are registered users, such as usernames, passwords, full names, and resources that said users have posted. Also, those who have accounts have full access to all approved and unapproved resources and have the capability to search for different files to be added to the list and/or share with other users.

Admin: Admin is a person/ team who is in charge of managing the website. The responsibilities of the admin include maintaining and troubleshooting the website, implementing security protocols, ensuring efficient user experience, and updating the website's contents. Admin also has authoritative power to approve or disapprove resources that users add to the website.

Resources: Resources are the contents added by the users in their account. A registered user can also pin a resource for easy access to the resource in the future. The list of resources are text files, images/video, online contents. The maximum size of the resources a registered user shall upload would be 1 GB.

- **Text files:** Text files include files with extensions .pdf, .doc, and .txt.
- **Images:** Image content includes files with extension .jpg, .jpeg and .png.
- **Video:** Video content will be stored as a link.
- **Audio:** Audio content will be stored as a link.
- **Online Contents:** Online contents include the online resources such as link to webpage, YouTube videos.

Unapproved Resources: Unapproved resources are the resources that are yet to be approved by the admin or the resources that failed to acquire permission from the admin to be added to the website. The criteria for resources to be approved include the relevancy of the content and if it is related to the course. The unapproved resources are listed as approved resources once they receive approval. Suppose the resources are unsuccessful in gaining permission. In that case, the resource is removed from the website, and the respective user of the resource is informed via the notification panel of the website.

Approved Resources: Approved resources are the resources that are approved based on the approval criteria. Once the resources are approved by the admin, the users can add the content to the list and share it with other users with the generated link.

Directory and Directory Tree: A directory is a folder that contains resources inside. A directory tree is the structure to organize the contents in the directory. Registered users can create multiple directories and subdirectories dedicated to different courses and topics. Registered users can use the default layout or create it for note organization. Registered users can label directories as professor's name, course name, or course abbreviation and number (CSC648, for example). Registered users can share the directories and subdirectories with other users with the generated link.

Comment: Comments are the remarks that users leave for uploaded resources. Only registered users shall be able to leave a comment on a resource. Registered users can leave one or multiple comments on their own resources, shared resources or publicly accessible resources. The posted comments are verified if appropriate by the admin. If comments are not appropriate, the admin can delete the comment from the resource.

Friend: Friend(s) are other registered users who are added as ‘Friend’ by a registered user. Adding other registered users as friends helps find users faster by applying filters while searching. A registered user has options to add/remove friends, and to list friends as ‘Favorite.’

Note: Note(s) are the record of brief descriptions and/or thoughts registered users can add to their accounts. A panel is available for registered users to add notes to their accounts. The associated note panel contains features where a registered user can type and save notes on the website.

Prioritized Functional Requirements

Priority 1

General User

1.1. A general user shall be able to create one or more accounts.
1.2. A general user shall be able to sign up with a unique email address.
1.3. A general user shall be able to sign up with a non-unique password.
1.4. A general user shall be able to see if all requirements for creating a password are met.
1.5. A general user shall be able to sign up with a non-unique username.
1.6. A general user shall be able to read the terms and conditions.
1.7. A general user shall be able to agree to the terms and conditions.

Registered User

2.1. A registered user shall have at least one account.
2.3. A registered user shall be able to view their account information in the Account page.
2.4. A registered user shall be able to temporarily unhide account password information in the *Account* page. By default, when a registered user visits the *Account* page, the password would be hidden for security and password.
2.5. A registered user shall be able to change their account username.
2.6. A registered user shall be able to change their account email.
2.7. A registered user shall be able to change their account password.
2.8. A registered user shall be able to upload any supported file type to their account's cloud storage.
2.10. A registered user shall be able to navigate through files with a directory tree.
2.11. A registered user shall be able to organize files alphabetically through a directory tree.
2.12. A registered user shall be able to view the current directory from a *Files* panel.
2.13. A registered user shall be able to create directories.
2.14. A registered user shall be able to create text files.
2.15. A registered user shall be able to create bookmarks of web URLs.
2.16. A registered user shall be able to pin the resources for easy access in the future.
2.17. A registered user shall be able to view supported files on the website.
2.18. A registered user can view the contents of their files on a *Display* panel.
2.19. A registered user can view the contents of their saved web pages on a *Display* panel.

2.21. A registered user shall have a place to type and save a note for each resource.
2.22. A registered user may have an assistant AI chatbot.
2.24. A registered user shall be able to comment on files.
2.25. A registered user shall be able to delete/move resources to a trash bin.
2.26. A registered user shall be able to delete resources from a trash bin permanently.
2.27. A registered user shall be able to recover items from a trash bin.
2.28. A registered user shall be able to download files onto their computer.
2.30. A registered user shall be able to share their directories by configuring the list as ‘public’.
2.31. A registered user shall be able to configure permissions of their shared directories/folders. The permission allows other users to either only view the list or download the list.
2.33. A registered user shall be able to view other directories shared publicly.
2.35. A registered user shall be able to download other shared directories onto their computer.
2.40. A registered user shall be able to add other registered users as friends.
2.41. A registered user shall be able to remove other registered users from friend-list.
2.42. A registered user shall be able to mark other registered users as their favorite friend.
2.45. A registered user shall be able to configure visibility of their resources at any time. The visibility can be either public or private.
2.46. A registered user shall be able to message another registered user.
2.47. A registered user shall be able to create a group chat to message multiple other registered users.
2.48. A registered user shall be able to search other registered users whose profile is set public.
2.49. A registered user shall be able to search friends.
2.50. A registered user shall be able to search pinned resources.
2.51. A registered user shall be able to clear notes added to the resource.
2.52. A registered user shall be able to clear comments added to the resource.

<u>Account</u>

3.1. Account shall have a certain amount of cloud storage.
3.2. Accounts shall require and store a username that is at least 6 characters.
3.3. Accounts shall require and store a real name tied to them, and the real name must have a first and last name.
3.4. Accounts shall require and store a password tied to them.

3.5. Accounts password shall be encrypted when stored in a database.
3.6. Accounts shall be created by one or more general users.
3.7. An account shall be owned by one registered user.

Admin

4.1. Admin shall be able to access all resources, approved or not
4.2. Admin shall be able to delete any resources present on the website
4.5. Admin shall be able to delete/ban any user for any reason (breaking of rules/TOS, inappropriate username)
4.6. Admin shall be able to delete any inappropriate resources uploaded.
4.8. Admin shall check if comments on resources are appropriate.
4.9. Admin shall delete zero or many inappropriate comments on one or many resources.

Resources

5.1. Resources shall be uploaded by a registered user.
5.2. Resources shall be organized in an alphabetical order.
5.3. Resources such as text files shall be able to be created by a registered user.
5.4. Resources shall be viewed by a registered user.
5.5. Resources shall be commented by registered users.
5.6. Resources shall be moved by a registered user to a trash bin.
5.7. Resources shall be able to be deleted permanently by a registered user from a trash bin.
5.8. Resources shall be recovered by a registered user from a trash bin.
5.9. Resources shall be downloaded on the computer by a registered user.
5.10. Resources shall have comments from other registered users.
5.11. Resources shall be able to be accessed by admin.
5.12. Resources shall be able to be deleted by admin.
5.13. Resources shall be deleted if deemed inappropriate by the admin.
5.15. Resources shall have comments verified by admin.
5.16. Resources shall be pinned by a registered user.
5.17. A resource shall have a note created by a registered user.
5.18. Resources shall have configured visibility by a registered user.
5.19. A pinned resource shall be searched by registered users.

Directory and Directory Tree

8.1. Directory trees shall be navigated by a registered user.
8.2. Directory trees shall have files organized in alphabetical order.
8.3. Directories shall be created by a registered user.

8.5. Directories shall be able to be shared by registered users by configuring them to be ‘public.’
8.6. Directories shall have configured permissions added by a registered user.
8.8 Directories shared by configuring it to ‘public’ shall be able to be viewed by a registered user.
8.10. Directories shall be able to be downloaded by other registered users onto their own computer.

Comment

9.1. Comments shall be added on resources by registered users.
9.2. Comments on resources shall be verified by admin.
9.3. Comments shall be deleted by the admin, if deemed inappropriate.

Friend

10.1. A friend shall be added by registered users.
10.2. A friend shall be removed by registered users.
10.3. A friend shall be added as a favorite by registered users.
10.4. A friend shall be searched by registered users.

Note

11.1 A note shall be typed and saved by a registered user for each resource.
11.3. Notes shall be cleared by registered users at any time.

Priority 2

General User

1.8. A general user shall be able to resend an account verification email.

Registered User

2.29. A registered user shall be able to share their directories via generated link.
2.32. A registered user shall be able to view other directories shared via link.
2.34. A registered user shall be able to download other shared directories into their account.
2.36. A registered user shall be able to search public directories.
2.37. A registered user shall be banned for uploading one or many inappropriate resources by admin.
2.38. A registered user shall not be able to view unapproved resources.
2.39. A registered user shall not be able to share unapproved resources.

2.53. A registered user shall be able to disable comments in the resource.
2.54. A registered user shall be able to delete a group chat they created.

Admin

4.3. Admin shall be able to change an unapproved resource to an approved resource.
4.4. Admin shall be able to change approved resources to unapproved resources.
4.7. Admin shall be able to view unapproved resources.

Unapproved Resources

6.1. An unapproved resource shall only be viewed by an admin.
6.2. An unapproved resource shall not be viewable by users until approved.
6.3. An unapproved resource shall be able to be converted to an approved resource by admin.
6.4. Unapproved resources shall not be able to be shared until an admin approves it.

Approved Resources

7.1. Approved resources shall be able to be viewed by admins and users.
7.2. Approved resources shall be able to be converted back to unapproved resources by the admin.
7.3. Approved resources shall be able to be shared via a generated link.

Directory and Directory Tree

8.4. Directories shall be able to be shared by registered users to other registered users via generated link.
8.7. Directories shared via generated links shall be able to be viewed by a registered user.
8.9. Directories shall be able to be downloaded by other registered users onto their own account.
8.11. Directories of other users registered configured as public shall be able to be searched by registered users.

Comment

9.5. Comments shall be able to be cleared by a registered user at any time.
9.6. Comments shall be disabled by a registered user at any time.

Priority 3

General User

1.9. A general user shall be provided a user manual covering basic features.

Registered User

2.2. A registered user shall be able to interact with a skippable tutorial sequence.
2.9. A registered user shall be able to cancel an upload that is in progress.
2.20. A registered user shall be able to change the color theme of the website.
2.23. A registered user shall be able to highlight text on pdf files.
2.43. A registered user shall be able to edit their posted comments at any time.
2.44. A registered user shall be able to edit their saved notes at any time.

Resources

5.14. Resources such as pdf files shall be highlighted by registered users.

Comment

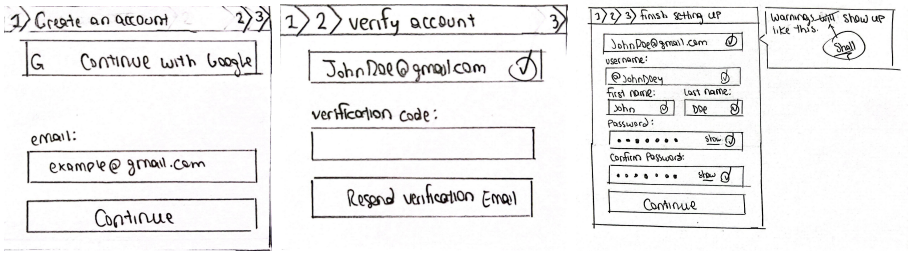
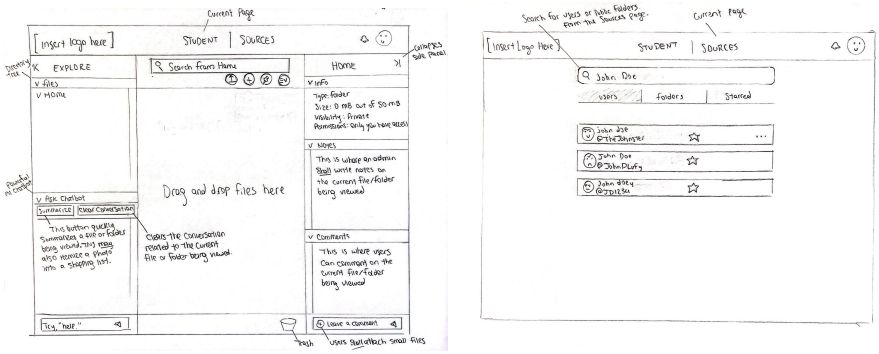
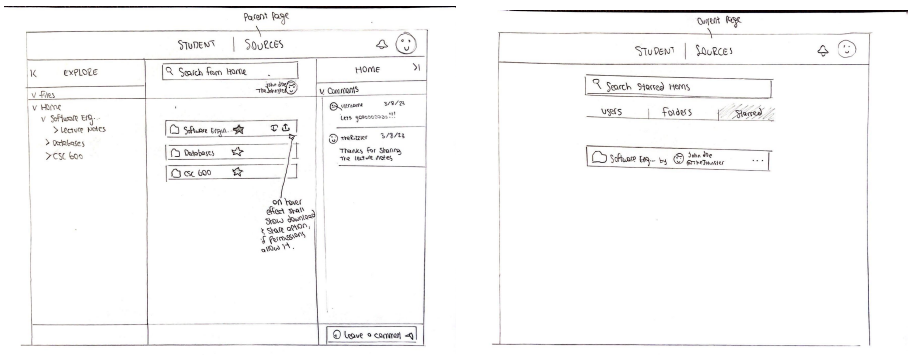
9.4. Comments shall be able to be edited at any time by the registered user who posted it.

Note

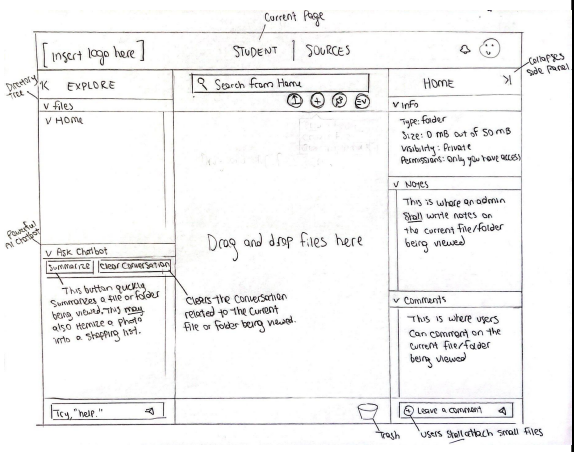
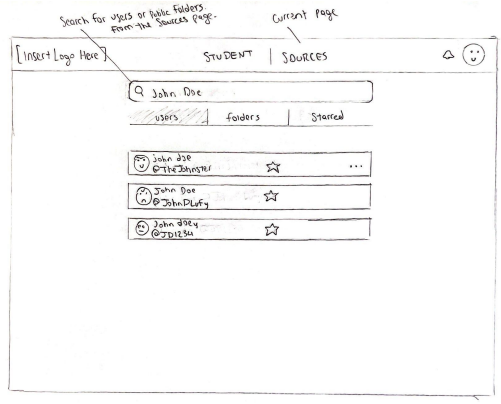
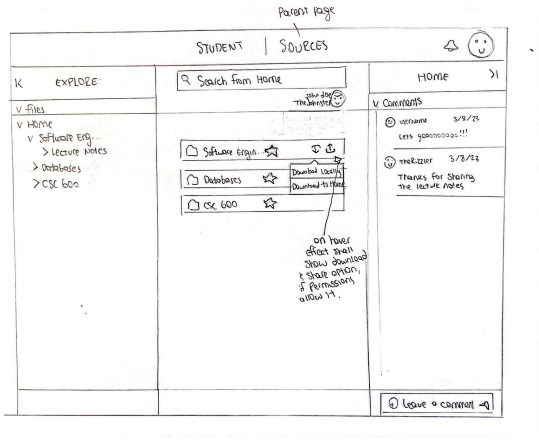
11.2. A note shall be able to be edited at any time by the registered user who saved it.

UI Mockups and Storyboards

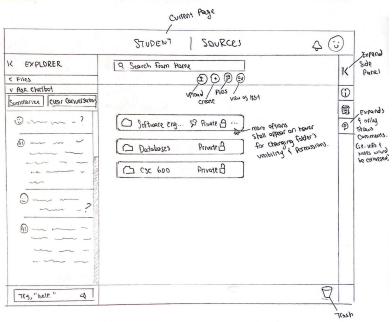
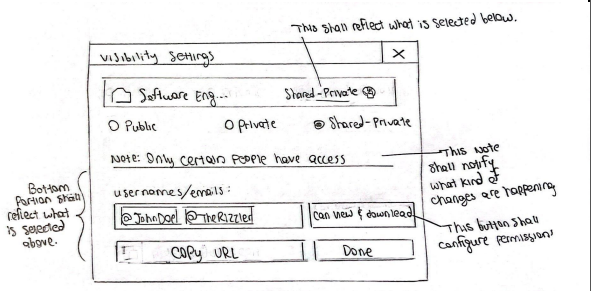
Use case 1: Struggling with grades (Aaron, 19, 2nd year in college)

UI Mockup	Description
	Aaron goes to the signup page of StudentSources, where he registers an account. After he submits a valid email, he finishes setting up his account.
	Aaron arrives at the landing/home page after signing in. Aaron goes to the sources tab to search for his friend who has the lecture notes for his software engineering course.
	Aaron clicks on his friend's account, after which he is brought to his friend's home page, where he can view all of his friend's publicly made folders. Aaron leaves a star on the folder containing the lecture notes, so he can quickly access them from his starred list.

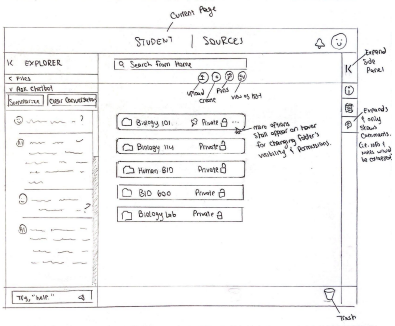
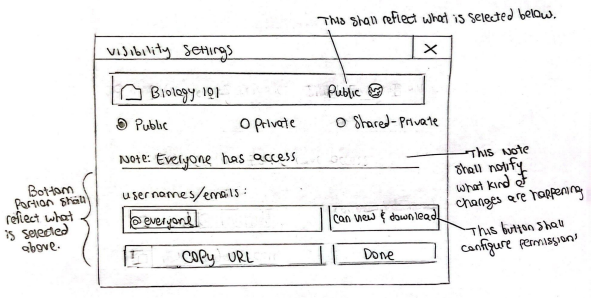
Use case 2: About to graduate (Sydney, 23, 4th-year college)

UI Mockup	Description
	Sydney arrives at her landing/home page after signing up and verifying her account. She wants to search for her friends' lecture notes, so she does not upload or create anything herself.
	Sydney navigates to the sources page to search for her friend, who was a previous student of a course Sydney is currently taking. The ‘users’ section is already auto selected upon arrival, so she begins searching from a list of StudentSources users.
	Sydney navigates to her friend’s home page by clicking on her username, and downloads the public folder containing relevant notes to her local computer, so that she can review them offline. Afterwards, she leaves a comment on her page to thank her friend for the lecture notes.

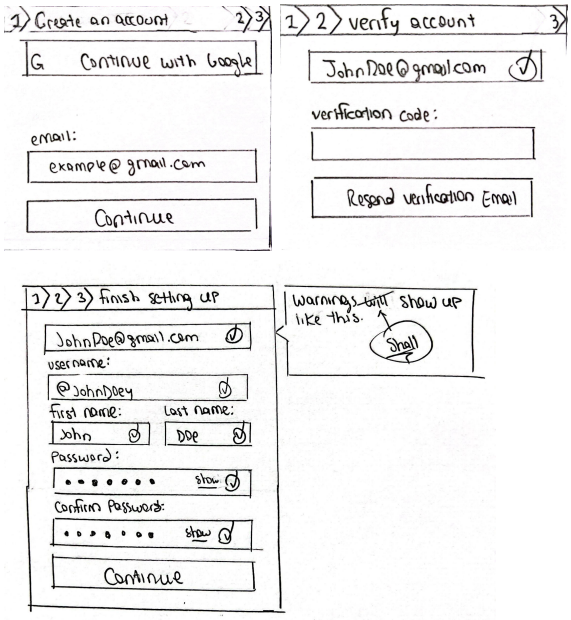
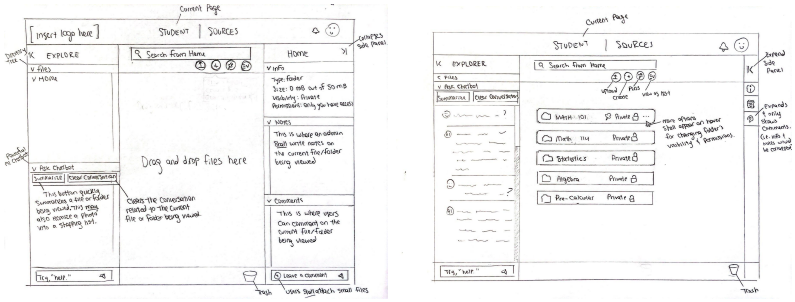
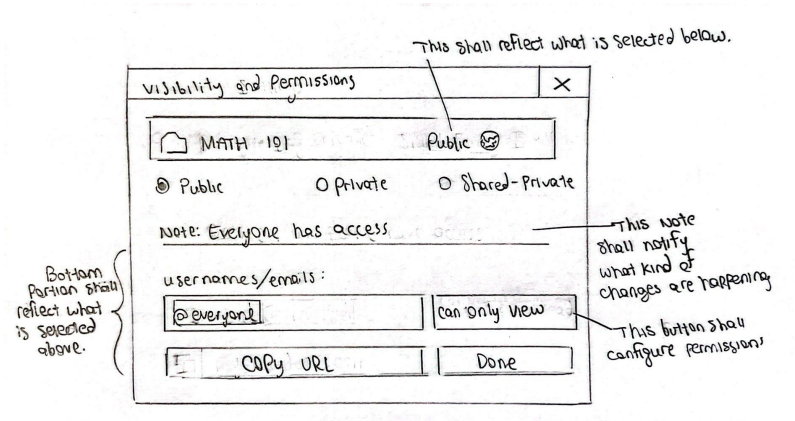
Use case 3: A friend needs help with studying (Darren, 22, 3rd-year college)

UI Mockup	Description
	Darren has a list of study materials for his Software Engineering class on the StudentSources website. He wants to keep his folder of resources private; not searchable on the sources page, yet he wants his friend to have access to it, and be able to download it.
	Darren configures his private folder to a shared-private folder in a pop-up window, where only a select few of valid StudentSources users have access to view and download his folder. With this feature, he can safely share his resources, while remaining anonymous from the general public.

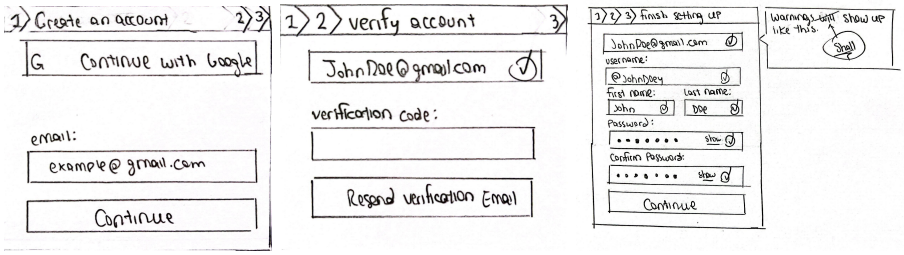
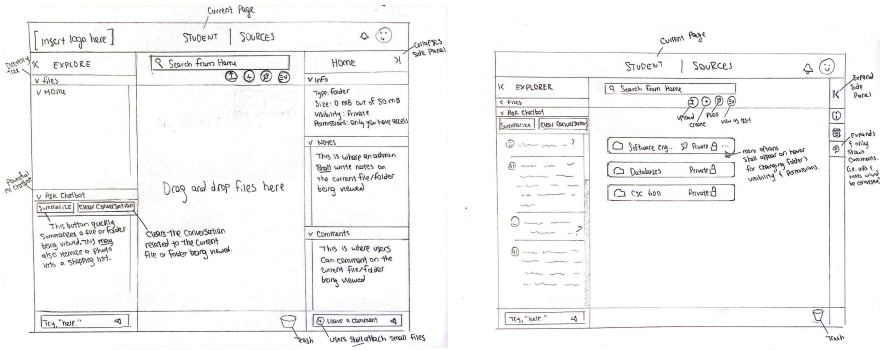
Use case 4: Helping students (Dave, 42, Professor of Biology)

UI Mockup	Description
	Dave is a professor teaching several subjects in Biology, and wants to make it easy for his students to access his curated list of biological research and documentation.
	Dave sets his folders of various Biological subjects to ‘public’ visibility in a pop up window, because he wants everyone, including those who cannot afford to go to college, to easily view and download his carefully curated material.

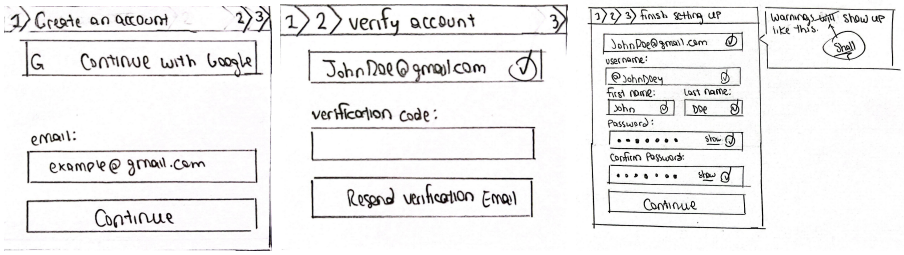
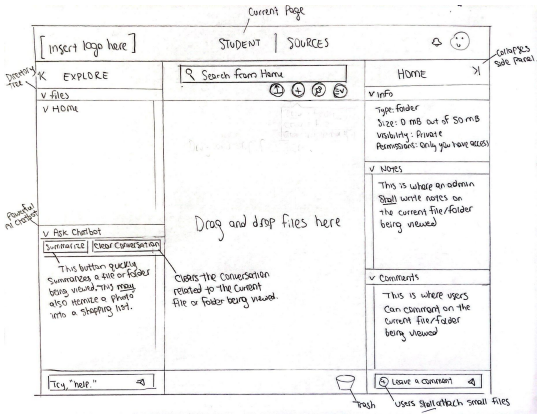
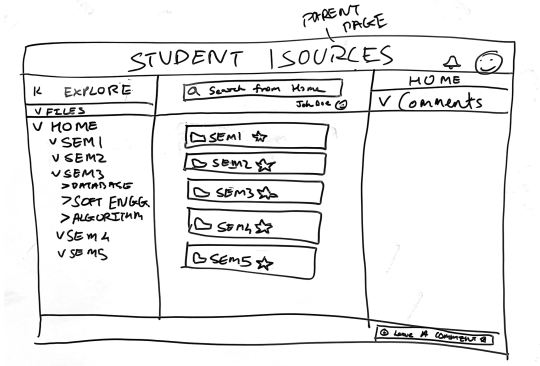
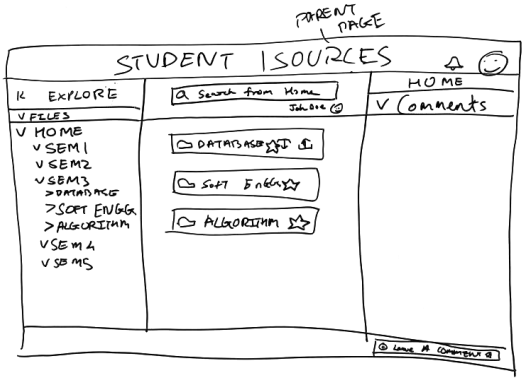
Use case 5: Help for tutoring (Milena, 32, Tutor)

UI Mockup	Description
	Milena is a middle school tutor of various mathematical subjects, and wants to create a list of resources. She already has some curated folders of resources stored on her personal cloud account and wishes to make these folders publicly visible so that her students can access them. But she is uncomfortable with the idea of having middle schoolers accessing her personal cloud account. So she sets up an account on StudentSources to have the folder of study material publicly accessible to her students.
	Milena closes some panels before dragging and dropping some folders containing study material onto her home page, where it is automatically set to private. This means that nobody can see these files on the sources page, and that it has not been shared with anyone else.
	Milena sets the folder to public in a pop-up window named “visibility and permissions”, so that anybody can search it from the sources page, but sets the permissions to view-only, so that they cannot download it.

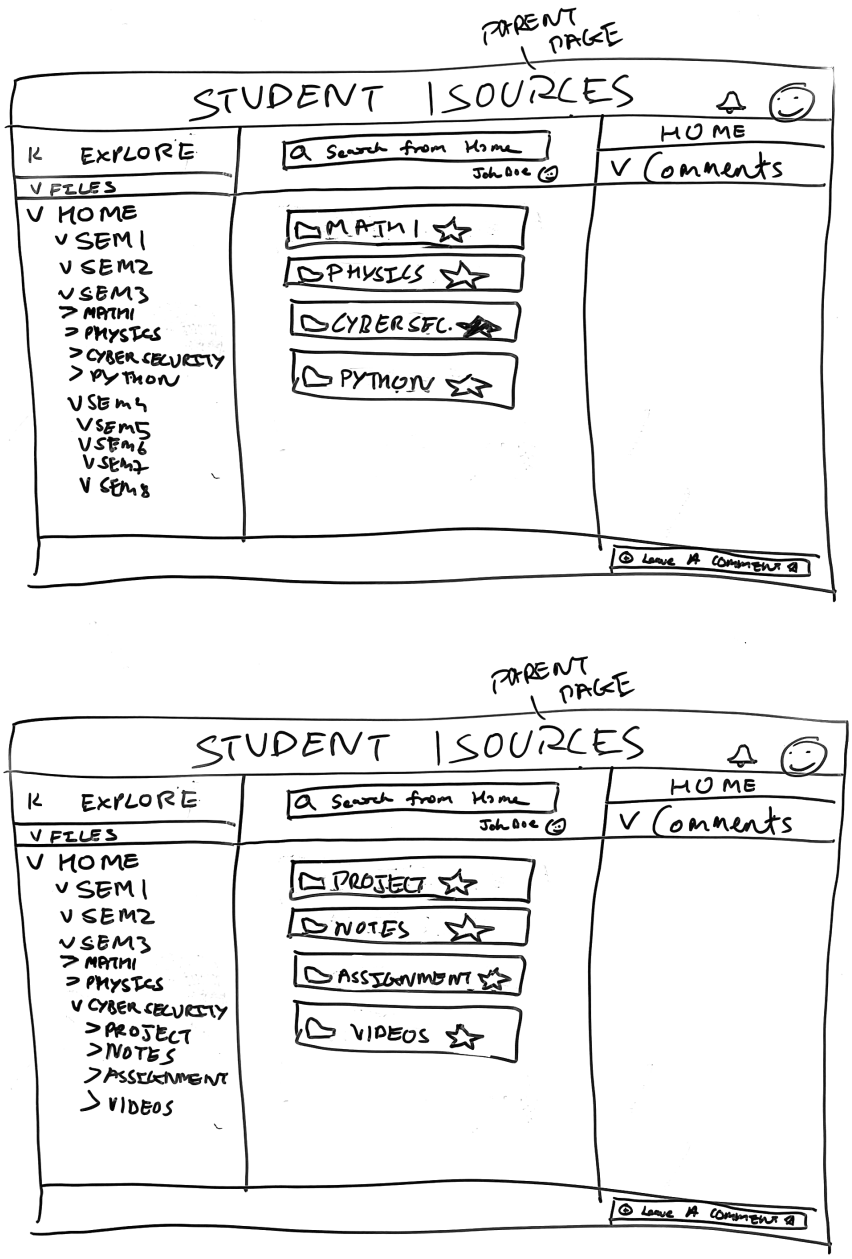
Use case 6: Technology Troubles (Marc, 20, Organized student)

UI Mockup	Description
	After losing all of the notes because of a broken laptop. For this semester Marc first goes to the signup page of StudentSources, where he registers an account. After he submits a valid email, he finishes setting up his account.
	Marc arrives at the landing/home page after signing in. Initially he has no notes or assignments at the start of the semester. He creates separate folders for each course by clicking on the create button and stores his notes here which he is able to access across different computers.

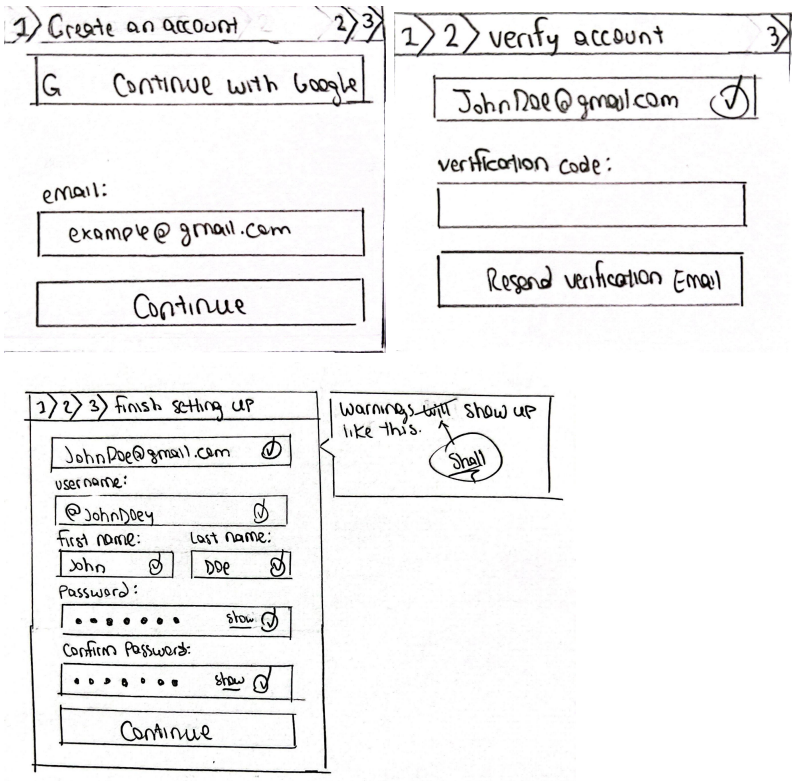
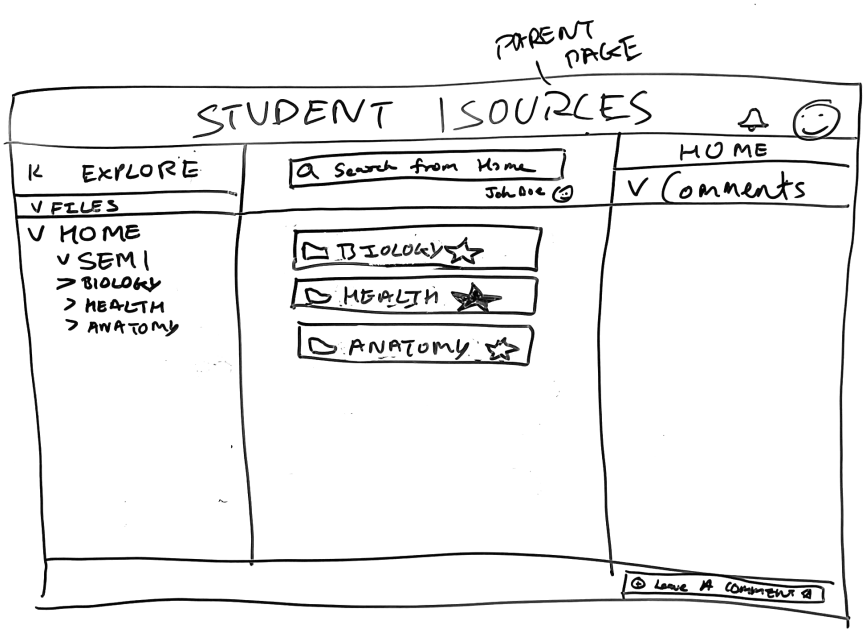
Use case 7: Attending many classes (Zander, 21, college student)

UI Mockup	Description
	Zander goes to the signup page of StudentSources, where he registers an account. After he submits a valid email, he finishes setting up his account.
  	Zander, being overwhelmed by the courses, organizes the courses in a well manner with folders. First he starts creating a folder for each semester. Then in each semester folder he organizes different courses. Hence whenever he is stuck he can go back to the previous semester to learn again!

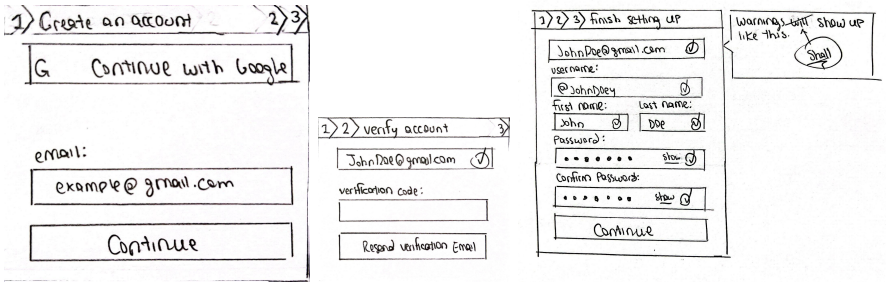
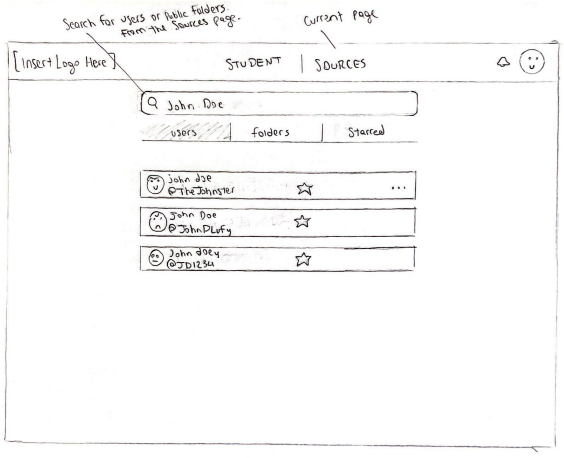
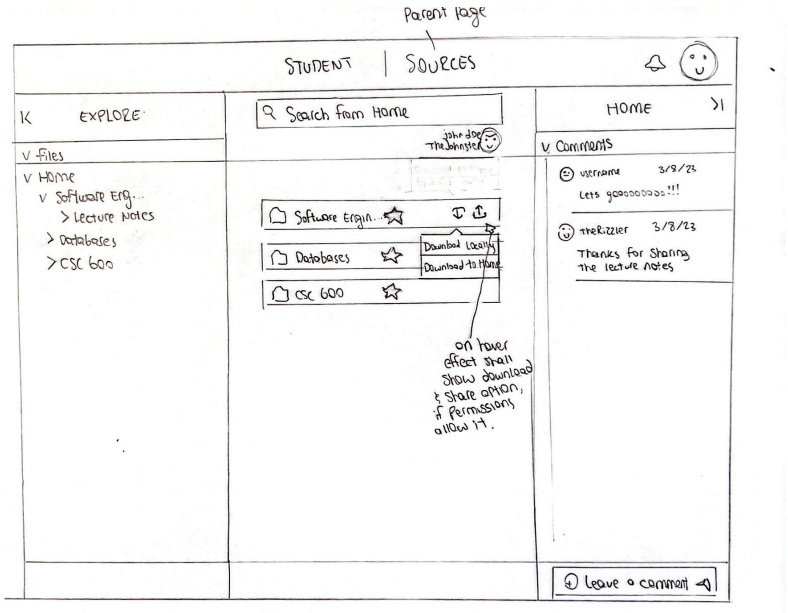
Use case 8: Landing A Job (Donovan, 26, recent graduate, looking for jobs)

UI Mockup	Description
	Donovan, is a recent SFSU graduate and is now looking for a job. He had the good habit of storing all of his notes in Student sources. So now he is looking for a job in the cyber security niche. He logins and goes to the cyber security folder to access all the project, notes, assignments and videos.

Use case 9: Overwhelming Course Load Candace — (26, overwhelming course load)

UI Mockup	Description
	Candace goes to the signup page of StudentSources, where she registers an account. After she submits a valid email, she finishes setting up her account.
	Candace, a nurse major & a transfer student. Uses student sources to store her notes for different subjects in an organized way and is able to keep up with classes and gets good GPA in her first semester

Use case 10: Imposter Syndrome (20, considering dropping out or changing majors)

UI Mockup	Description
	Jacob goes to the signup page of StudentSources, where he registers an account. After he submits a valid email, he finishes setting up his account.
	Jacob is unable to keep up with the professor's speed to take up notes. He searches for a classmate's name and taps on his name.
	On clicking it opens up the friend's folder and he is able to access the public folder and is able to revise and study well.

High level database architecture and organization

This high level conceptual database design will be based on one of the unique features StudentSources provides, which is the exclusivity to SFSU students and being able to have a dedicated community for SFSU study resources.

Database Organization

Database Requirements:

1. A registered user shall be able to upload zero or many resources.
2. A registered user shall add zero or many comments on their own resources.
3. A registered user shall comment on many other registered users' resources.
4. A registered user shall add zero or many other registered users as friends.
5. A registered user shall remove zero or many other registered users from friend-list.
6. A registered user shall add zero or one note for each resource.
7. A registered user shall set permissions to ‘downloadable’ of many of their shared lists.
8. A registered user shall set permissions to ‘view only’ of many of their shared lists.
9. A registered user shall set visibility as ‘public’ of many of their resources.
10. A registered user shall set visibility as ‘private’ of many of their resources.

Entities:

1. registeredUser
2. userResource
3. resourceComment
4. userFriend
5. sharedList
6. userNote

Entity Attributes and Domains:

1. registeredUser
    a. userID (primary key) = INT, numeric, unique, and not null

    b. username = VARCHAR(32), not null, alphanumeric, can be upper and lowercase, must be unique

    c. email = VARCHAR(50), not null, alphanumeric, must contain valid email with a @

d. password = VARCHAR(32), not null, alphanumeric, will be encrypted before storing, must have requirements specified by website

e. formalName = VARCHAR(32), first and last name of user, can only contain letters

f. userImage = BLOB, user profile image saved as binary data

g. sharedListID (foreign key) = INT, numeric, references sharedList's sharedListID

h. resourceID (foreign key) = INT, numeric, references userResources' resourceID

i. commentID (foreign key) = INT, numeric, references resourceComment's commentID

j. noteID (foreign key) = INT, numeric, references userNote's noteID

2. userResource
    a. resourceID (primary key) = INT, numeric, unique, not null, auto incremental

    b. title = VARCHAR(32), not null, alphanumeric, must be unique to prevent confusion of resources.

    c. description = TEXT, can contain anything TEXT data type can store including letters, numbers, special characters. Holds a string with a max length of 65,535 bytes.

    d. uploadDate = DATE, Format: YYYY-MM-DD. The supported range is from '1000-01-01' to '9999-12-31'

    e. courseID = VARCHAR(12), alphanumeric, will be stored as 3-4 letters following 3 numbers, for example: CSC101

    f. professorName = VARCHAR(32), alphanumeric, can only contain letters and hyphens

g. isPrivate = TINYINT(1), equivalent to boolean data type, can only contain 0 or 1, tracks if the resource is private or public

h. userID (foreign key) = INT, numeric, references registeredUser's userID

i. commentID (foreign key) = INT, numeric, references resourceComment's commentID

j. noteID (foreign key) = INT, numeric, references userNote's noteID

userResource entity contains child entities which define the format of resources that users store. The child entities with their respective attributes are as follows:

i. textFile
   a. size = INT, numeric
   b. extension = VARCHAR(10), alphanumeric, can contain letters and a special character like '.'

ii. image
   a. size = INT, numeric
   b. extension = VARCHAR(10), alphanumeric, can contain letters and a special character like '.'

iii. video
   a. url = VARCHAR(200), alphanumeric, can contain letters, numbers, and special characters such as ; , / ? : @ & = + $ - _ . ! ~ * ' ( ) #

iv. audio
   a. url = VARCHAR(200), alphanumeric, can contain letters, numbers, and special characters such as ; , / ? : @ & = + $ - _ . ! ~ * ' ( ) #

v. webpage
   a. url = VARCHAR(200), alphanumeric, can contain letters, numbers, and special characters such as ; , / ? : @ & = + $ - _ . ! ~ * ' ( ) #

3. resourceComment
   a. commentID (primary key) = INT, numeric, not null, unique

b. comment = TINYTEXT, alphanumeric, can contain anything TINYTEXT data type can store including letters, numbers, special characters. Holds a string with a max length of 255 characters.

c. commentDate = DATE, Format: YYYY-MM-DD. The supported range is from '1000-01-01' to '9999-12-31'

d. resourceID (foreign key) = INT, numeric, references userResource's resourceID field, creating a relationship between the resource and the comment that was posted on the resource.

e. userID (foreign key) = INT, numeric, references registeredUser's userID field, creating relationship between the comment and the user who commented it.

4. userFriend
    a. UID1 (primary key and foreign key) = INT, numeric, not null, references registeredUser's userID
    b. UID2 (primary key and foreign key) = INT, numeric, not null, references registeredUser's userID

5. sharedList
    a. sharedListID (primary key) = INT, numeric, not null

    b. url = VARCHAR(200), alphanumeric, can contain letters, numbers, and special characters such as ; , / ? : @ & = + $ - _ . ! ~ * ' ( ) #

    c. isDownloadable = TINYINT(1), equivalent to boolean data type, can only contain 0 or 1, tracks if the resource is view only or is downloadable by other registered users

    d. userID (foreign key) = INT, numeric, references registeredUser's userID

    e. resourceID (foreign key) = INT, numeric, references userResource's resourceID

6. userNote
    a. noteID (primary key) = INT, numeric, not null,

b. note = TINYTEXT, alphanumeric, can contain anything TINYTEXT data type can store including letters, numbers, special characters. Holds a string with a max length of 255 characters.

c. userID (foreign key) = INT, numeric, references registeredUser's userID

d. resourceID (foreign key) = INT, numeric, references userResources' resourceID

Entity Relationships:

1. One-to-Many (1:M), registered user to resources
    a. A resource shall be owned/posted by **only one** user.
    b. A user shall be able to post **many** resources.

2. One-to-Many (1:M), registered user to resource comments
    a. A comment shall be posted by **only one** user.
    b. A user shall be able to post **many** comments on a resource.

3. One-to-Many (1:M) registered user to user friend
    a. A friend shall be associated with **only one** user. For example, when user A sends a friend request to user B and the request is accepted, the connection shall be only between those two users.
    b. A user shall be able to accept **many** friend requests and become friends.

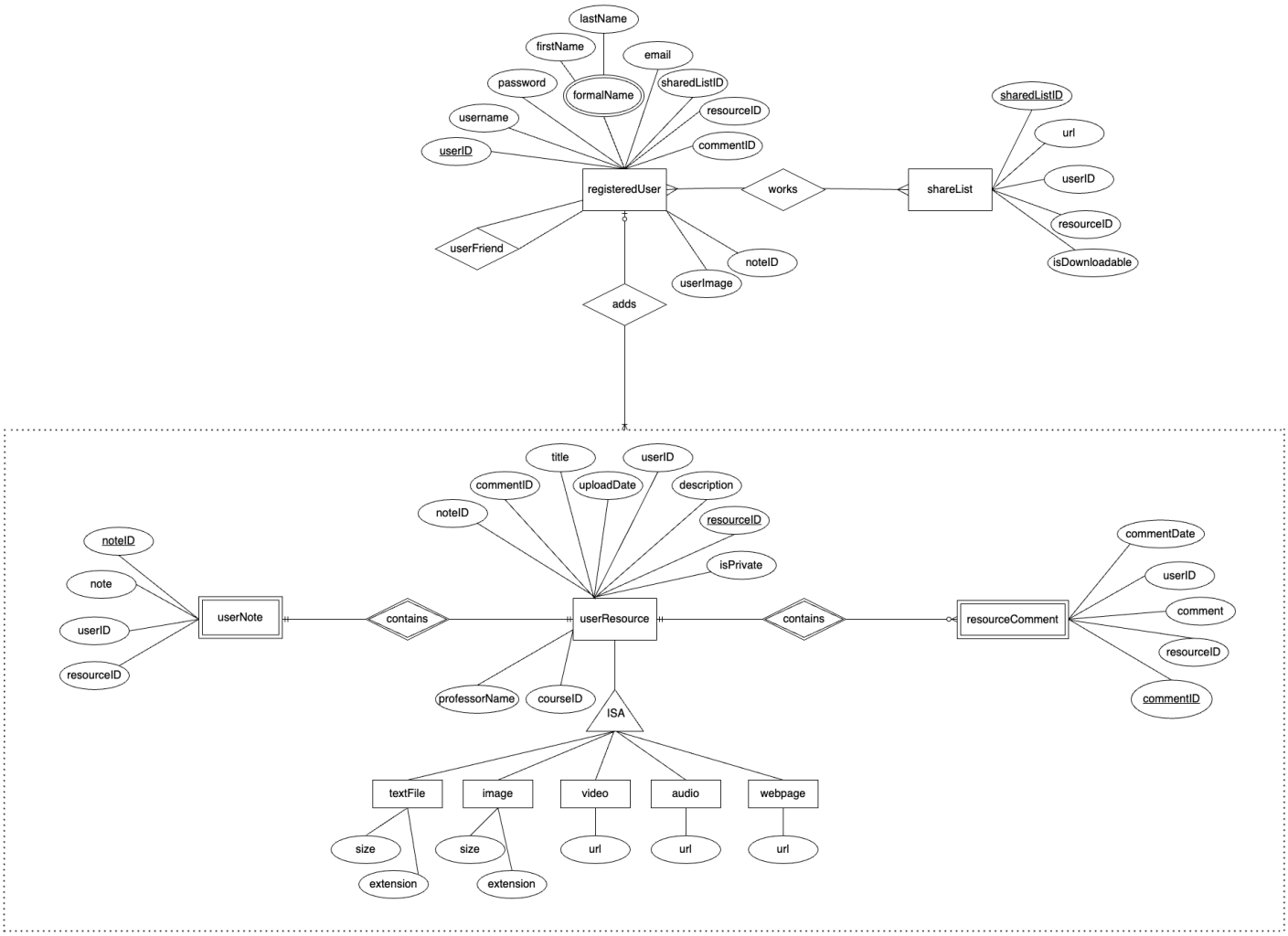
4. Many-to-Many (M:M) registered user to shared list
    a. A registered user shall work on **many** shared lists.
    b. A shared list shall be worked on by **many** registered users.

5. One-to-One (1:1) registered user to note
    a. A note shall be associated with **only one** resource.
    b. A resource shall be able to contain **only one** note.

6. One-to-Many (1:M) resource to resource comments
    a. A comment shall be associated with **only one** resource.
    b. A resource shall be able to contain **many** comments.

7. One-to-One (1:1) resource to user note
    a. A resource shall have **only one** user note.
    b. A user note shall be added to **only one** resource.

Entity Relationship Diagram (ERD):



Database Management System (DBMS):

The DBMS we chose for database management is MySQL because we are familiar with its syntax; in addition, it is simple and efficient.

Media Storage

The multimedia contents will be stored in a mountable file system. The data format for images are jpeg and png, for audio files are mp3 and for video files are mp4.

Search/filter architecture and implementation

The algorithm used to search for registered users in the database starts in the user repository in the searchUsersByUsername method with the SQL statement “SELECT * FROM user WHERE user LIKE :query OR first_name LIKE :query OR last_name LIKE :query”. This statement, which will select entities with identical values in the user, first_name, or last_name attributes, is recognized as a query due to Spring Boot’s Query annotation and then stored into a string parameter called query. The Query is then modified to handle white spaces and then stored into another string variable called newQuery, which is then passed into searchUsersByUsername and returned as an User entry. After that, both the password and email of said user are encrypted via for loop and the entry is then returned if it exists and will return null if it is not.

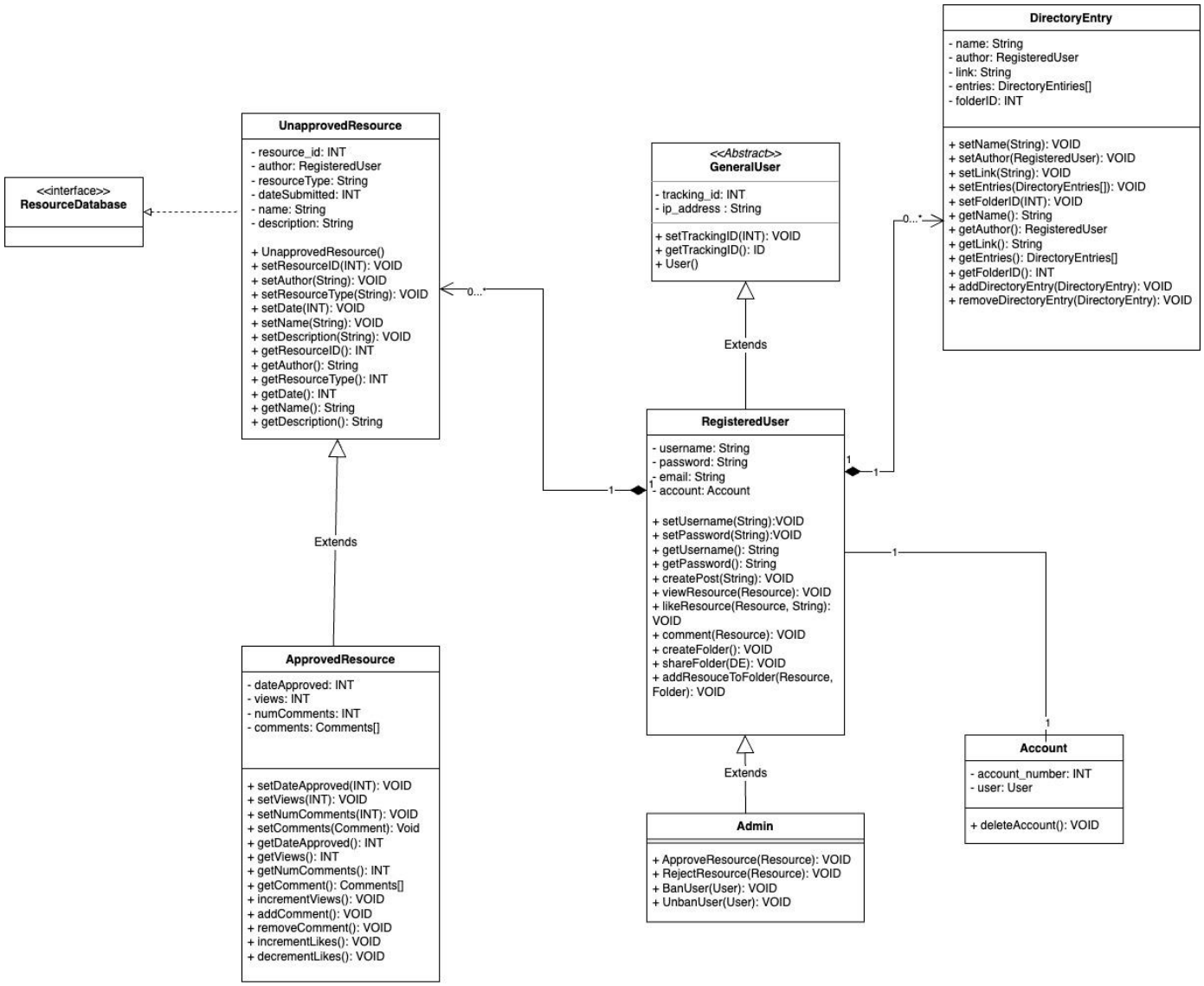
High Level APIs and Main Algorithms

Main Algorithms

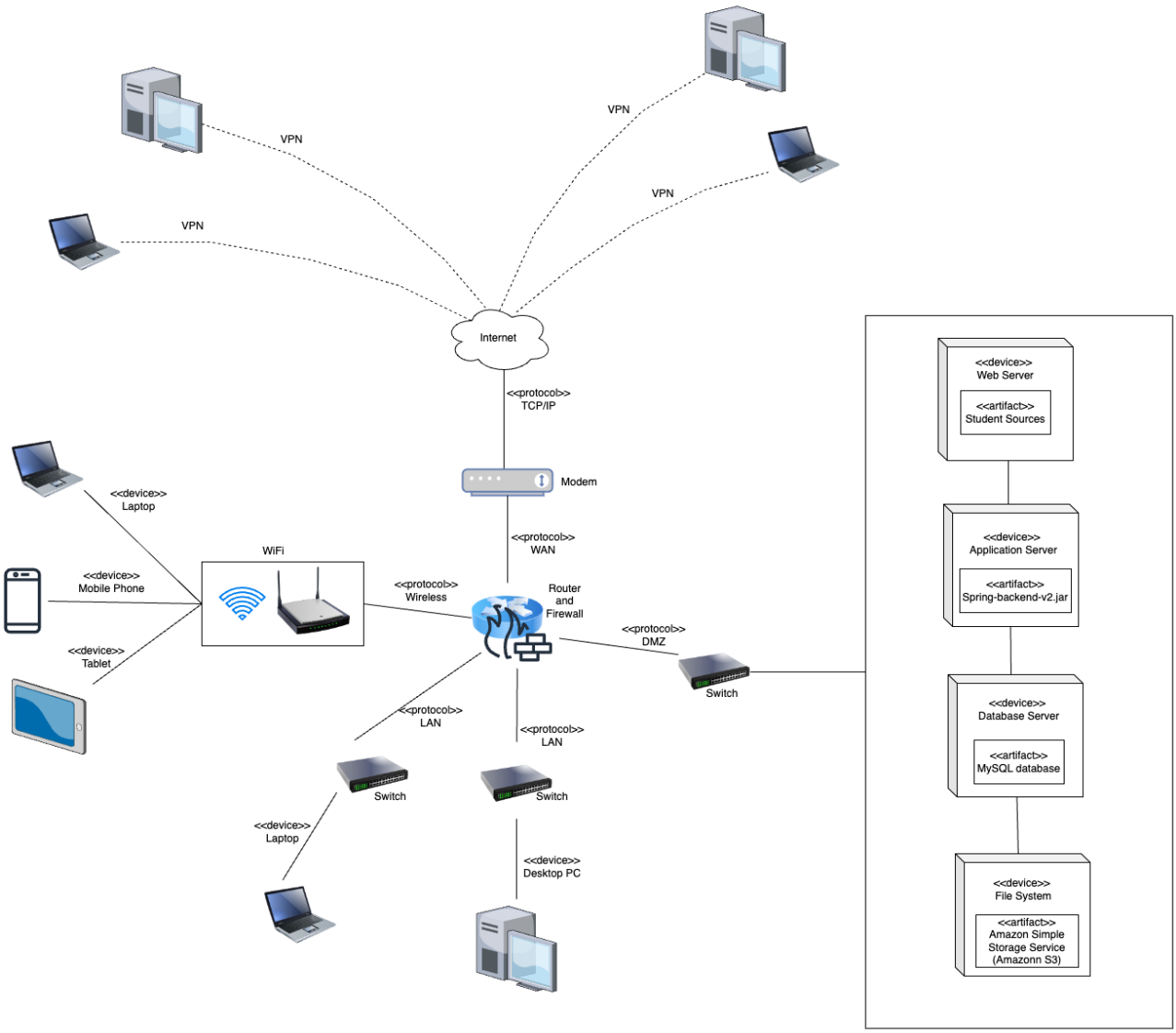
- **Sorting Algorithm:**
  A sorting algorithm will be used for getting the most useful documents first. There are a variety of metrics we can use for sorting such as views, impressions, etc. For example, a user can search through a professor's most viewed documents which will help students get a better understanding of the professor's course. By having a sorting algorithm, we can increase user retention by providing the most relevant documents first.

- **Filter Algorithm:**
  A filtering algorithm allows us to obtain documents of a specific category. Our application should be able to filter through documents by date, course, professor, and/or document type. For example, a user might want only documents that are relevant within the last 2 years or a user might want only documents from a specific professor. By having a filtering algorithm, users can navigate through documents in a convenient manner.

High Level UML Diagrams



High Level Application Network and Development Diagrams



Key Risks in Current Work

Skills Risks

The required skill for the product is knowledge in Java and JavaScript. The skill rating in our team of 7 resulted in an average of 4 on a scale of 1 to 5, with 5 being very familiar and 1 being never used for both Java and JavaScript. This result is a good indication of a member's familiarity with the languages. However, it is understandable that there would be times where members might need help with those languages. In those scenarios, the team would take the time to learn what help the member needs and help the member by providing time and resources.

Schedule Risks

The team is composed of students who are Graduate students and seniors in their Undergraduate Studies. In addition, members are actively involved in extracurricular activities such as school clubs, research, and so on. Apart from these, members dedicate their time and effort in income-generating jobs. At a glance, it can be considered that members have busy schedules and the schedules are bound to conflict. Therefore, to minimize the effect of schedule risk, outside of the regular class time, every member commits an hour to meet on Monday. If members are unable to attend the meeting, they’re updated later in the Discord server by the team lead.

Technical Risks

Technical risk consists of using or referencing code from an unauthorized party. Since the team is a group of students learning to work on a big project, it can be presumed that the internet would be used as a source of reference; hence, there are possibilities where the referenced code might not have permission to be used. To resolve such cases, the team will confirm that the permission to use the code is received before incorporating the code in the project.

Teamwork Risks

In a project that depends heavily on team collaboration, the presence of teamwork risk is inevitable. There are few risks concerning teamwork. First, the decisions might not be unanimous among the members. Second, there could be communication issues such as miscommunication or lack of communication. Lastly, there could be unbalanced workload, meaning some teammates are not able to finish work on time and other members have to work to their maximum capacity or beyond to deliver the task by deadline.

In order to face those obstacles in teamwork, all sides related to a specific decision are explained and pros and cons are weighed; then, the decisions are made accordingly.

Therefore, every member comprehends the effects of the decision. Next, regarding communication issues, the members are encouraged to express their thoughts in team meetings, Discord server. To avoid miscommunication, the members are requested to explain the matter further so that the idea is conveyed clearly. And lastly, regarding unbalanced workload, the team lead ensures that tasks are assigned in a well adjusted manner. However, even though there is an attempt to balance workload, it might not execute perfectly as planned; thus, the efforts of each member are reflected on a contribution score table to make sure the members are credited for their efforts.

Legal/Content Risks

There are two possible risks related to legal/content risks. As a developer, we might encounter difficulties such as copyright issues while using other people/organizations' content. On the other hand, the legal consequences related to terms and conditions designated to end users might arise. To address content risk issues, the team will confirm that the contents being used are not copyrighted. Subsequently, to address legal issues, the terms and conditions document will be descriptive and transparent so that the users find it straightforward and easy to understand.

Project Management

In Milestone 2, we dive deeper into designing our product. The first step for the task management was to divide the Milestone into three parts: Milestone 2 documents, Vertical SW Prototype, and review. To accomplish the goal of completing Milestone on time, internal deadlines were set. The allocated time for Milestone 2 documents was a week; similarly, Vertical SW Prototype was two weeks, and for the review and preparation for submission, it was 2-3 days.

The technologies used for project management were Discord, Zoom, and Trello. The team's main line of communication was a Discord server dedicated to the team. Discord was used to post agendas, reminders, updates on members' progress, feedback, and requests for help. The secondary form of communication was Zoom, where the team met weekly to discuss matters requiring every member's input. The last project management tool used was Trello workspace. On the Trello workspace, different boards were created with checklists and deadlines, which allowed members to view the tasks and status at a glance. In addition, the reminder feature on Trello board was used to remind members of upcoming deadlines.

List of Team Member Contributions

Team Member	**Contribution**
Saru Shrestha	- Data Definitions - Prioritized Functional Requirements - High level database architecture and organization (documentation) - Deployment Diagram - Identify Key risks - Project Management (documentation) - Design logo (outsourced) - Draft Terms and Conditions document - Revise Milestone 2 documentation
Kilian Kistenbroker	- UI Mockups and Storyboards - High Level UML Diagrams - Frontend design and implementation - Vertical Prototype
Elias Abay	- Data Definitions - High level database architecture and organization: Search/filter architecture (documentation) - Vertical Prototype
Parth Panchal	- UI Mockups and Storyboards - High Level UML Diagrams - Vertical Prototype - Maintain Back-up GitHub Repository
Joshua Del Rosario	- High level database architecture and organization (documentation) - Database implementation in MySQL database
Wilson Yin	- High level database architecture and organization (documentation) - High Level APIs and Main Algorithms (documentation) - High Level UML Diagrams
Omar Thiongane	- Application Networks Diagram - Deployment Diagram

Team Member Contribution Score

Team Member	**Score**
Kilian Kistenbroker	9
Elias Abay	7
Parth Panchal	9
Joshua Del Rosario	9
Wilson Yin	9
Omar Thiongane	9

Scale: 1 to 10, where a score of 1 means “no work at all”, and score of 10 means “contributed to the maximum capacity”